OPTIONAL TASK

Name of Student : Anuja Vilas Wankhade

Domain : Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository Link: https://github.com/anujaw193-star/python-project-collection

---

1. Import Libraries:

```
[1]
✓ 0s
import numpy as np
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

2. Load Dataset:

```
[2]
✓ 0s
df = pd.read_csv("train (1).csv")
df.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD

5 rows × 81 columns

3. Handle Missing Values:

```
[5]
✓ 0s
# Fill numerical missing values with median
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Fill categorical missing balues with mode
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
  df[col] = df[col].fillna(df[col].mode()[0])
```

There no missing values in this dataset.

4. Prepare Features:

```
[6]
✓ 0s
X = df.drop("SalePrice", axis=1)
y = df["SalePrice"]
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

Shape of X: (1460, 80)
Shape of y: (1460,)
```

5. One-Hot Encoding

```
[7]
✓ 0s
from pandas.core.arrays import categorical
categorical_cols = X.select_dtypes(include=['object']).columns
X = pd.get_dummies(X, columns=categorical_cols)
print(X.head())
```

```
   Id  MSSubClass  LotFrontage  LotArea  OverallQual  OverallCond  YearBuilt  \
0   1          60         65.0     8450            7            5       2003
1   2          20         80.0     9600            6            8       1976
2   3          60         68.0    11250            7            5       2001
3   4          70         60.0     9550            7            5       1915
4   5          60         84.0    14260            8            5       2000

   YearRemodAdd  MasVnrArea  BsmtFinSF1  ...  SaleType_ConLw  SaleType_New  \
0          2003       196.0         706  ...           False         False
1          1976         0.0         978  ...           False         False
2          2002       162.0         486  ...           False         False
3          1970         0.0         216  ...           False         False
4          2000       350.0         655  ...           False         False

   SaleType_Oth  SaleType_WD  SaleCondition_Abnorml  SaleCondition_AdjLand  \
0         False         True                  False                  False
1         False         True                  False                  False
2         False         True                  False                  False
3         False         True                   True                  False
4         False         True                  False                  False

   SaleCondition_Alloca  SaleCondition_Family  SaleCondition_Normal  \
0                 False                False                  True
1                 False                False                  True
2                 False                False                  True
3                 False                False                 False
4                 False                False                  True
```

```
   SaleCondition_Partial
0                  False
1                  False
2                  False
3                  False
4                  False

[5 rows x 288 columns]
```

6. Feature Scaling:

```
[8]
✓ 0s
scaler = StandardScaler()
X = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
print(X.head())
```

```
         Id  MSSubClass  LotFrontage   LotArea  OverallQual  OverallCond  \
0 -1.730865    0.073375    -0.220875 -0.207142     0.651479    -0.517200
1 -1.728492   -0.872563     0.460320 -0.091886    -0.071836     2.179628
2 -1.726120    0.073375    -0.084636  0.073480     0.651479    -0.517200
3 -1.723747    0.309859    -0.447940 -0.096897     0.651479    -0.517200
4 -1.721374    0.073375     0.641972  0.375148     1.374795    -0.517200

   YearBuilt  YearRemodAdd  MasVnrArea  BsmtFinSF1  ...  SaleType_ConLw  \
0   1.050994      0.878668    0.514104    0.575425  ...       -0.058621
1   0.156734     -0.429577   -0.570750    1.171992  ...       -0.058621
2   0.984752      0.830215    0.325915    0.092907  ...       -0.058621
3  -1.863632     -0.720298   -0.570750   -0.499274  ...       -0.058621
4   0.951632      0.733308    1.366489    0.463568  ...       -0.058621

   SaleType_New  SaleType_Oth  SaleType_WD  SaleCondition_Abnorml  \
0     -0.301962     -0.045376     0.390293              -0.272616
1     -0.301962     -0.045376     0.390293              -0.272616
2     -0.301962     -0.045376     0.390293              -0.272616
3     -0.301962     -0.045376     0.390293               3.668167
4     -0.301962     -0.045376     0.390293              -0.272616

   SaleCondition_AdjLand  SaleCondition_Alloca  SaleCondition_Family  \
0              -0.052414             -0.091035             -0.117851
1              -0.052414             -0.091035             -0.117851
2              -0.052414             -0.091035             -0.117851
3              -0.052414             -0.091035             -0.117851
4              -0.052414             -0.091035             -0.117851
```

```
4              -0.052414             -0.091035             -0.11

   SaleCondition_Normal  SaleCondition_Partial
0              0.467651              -0.305995
1              0.467651              -0.305995
2              0.467651              -0.305995
3             -2.138345              -0.305995
4              0.467651              -0.305995

[5 rows x 288 columns]
```

7. Train- Test Split:

```
[9]
✓ 0s
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.33,
                                                    random_state=42)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(978, 288)
(482, 288)
(978,)
(482,)
```

8. Train Linear Regression Model

```
[10]
✓ 0s
mode = LinearRegression()
mode.fit(X_train, y_train)
print("Intercep:", mode.intercept_)
print("Coefficient:", mode.coef_)
```

```
Intercep: 180628.38587381854
Coefficient: [ 7.60708113e+01  8.25725942e+03  6.37413250e+02  7.69976784e+03
  9.77998482e+03  5.00909308e+03  7.58889388e+03  3.33202447e+03
  3.10712902e+03  6.64019348e+03  1.12852845e+03 -1.35498257e+02
  7.18195239e+03  4.34335681e+03  1.22345797e+04  5.53141688e+01
  1.33640002e+04  1.74744038e+03  2.23124204e+00  2.65445476e+03
  1.04998458e+03 -2.93324242e+03 -3.64743653e+03  4.00655488e+03
  1.43667984e+03 -5.93261747e+02 -2.29864600e+02  7.08298219e+03
  1.90037257e+03 -2.62297229e+02  7.61001619e+02  1.37741223e+03
  2.86213207e+03  3.76964497e+03  1.00390342e+03 -1.13983218e+03
 -1.07669752e+02 -1.63727356e+03  1.73180231e+03 -6.43644699e+02
 -4.59045756e+02  9.08331579e+01 -1.92130010e+03  1.92130010e+03
 -1.01236827e+03  1.01236827e+03 -6.17260802e+02  3.72572214e+02
  4.35732808e+02  4.00739694e+02 -1.28797811e+03  2.22516634e+03
 -2.01646033e+03  5.60749410e+02  3.63123693e+02 -3.63123693e+02
 -2.08496591e+02  2.12447598e+03 -1.37704646e+03 -1.13337677e+03
 -3.09744232e+02  4.22998278e+02  1.37347605e+03 -4.02808801e+03
  1.37077634e+03  3.90069662e+02  9.92062624e+02 -9.84374363e+00
 -2.84291479e+02  1.66211365e+00  2.74711664e+03 -3.46830508e+03
 -3.86584951e+02 -2.04551827e+03 -5.08671700e+01 -2.04965554e+03
 -3.85189468e+03  2.30592844e+03 -1.94324219e+03  5.17267832e+03
  6.43578138e+03 -3.13544343e+03  1.28525723e+02 -2.67002381e+02
  3.04219233e+02  7.58491820e+02  6.50390067e+03 -9.13255223e+00
  8.15665194e+02 -1.72955119e+03 -3.48549840e+02  1.42448646e+03
 -9.28833557e+02  8.51035106e+02 -1.92598897e+03  1.20184854e+02
  7.51630305e+01  3.14821929e+02  1.19831151e+03  1.47349057e+03
  1.68298505e+03  4.32009983e-12 -7.59679391e+03 -1.37292371e+03
  3.28707790e+02 -2.04636308e-12  8.06412618e+03 -1.56611905e+03
 -2.01814012e+01 -4.95795594e+03 -7.15987227e+03  5.75196405e+01
  1.57340641e+03  5.83653936e+03 -1.78542536e+03 -5.99123914e+02
 -4.60263505e+03 -2.07997070e+03 -2.23050315e+03  1.95046488e+02
 -3.54638991e+02  1.86451061e+02 -2.55575702e+02  1.83086969e+03
  2.35395231e+03 -1.62597059e+04  1.27091066e+03  8.07176548e-12
  1.53887291e+03  2.11228198e+02  6.54777807e+02  6.77600794e+02
```


```
  1.53887291e+03  2.11228198e+02  6.54777807e+02  6.77600794e+02
```


```
  1.80252200e+03  2.06867502e+03 -1.46866635e+02  2.22383534e+03
  4.43905307e+03 -2.96594704e+02 -1.47207711e+03  8.79000448e+02
  2.27373675e-13  1.10232244e+03  1.43250375e+03  3.19506558e+02
 -2.71564455e+02 -1.86538857e+03 -2.67386870e+03 -4.77761494e+02
 -9.16795659e+02  9.03802088e+02 -1.08617422e+03 -3.06444321e+01
 -2.96594704e+02  2.60693359e+03 -1.84981020e+03 -1.34489821e+02
 -7.03721222e+02 -8.41226387e+02 -3.00364234e+03 -6.68112172e+02
  1.63842113e+03  6.18269766e+02  2.92553341e+03 -1.19014339e+02
 -1.01242685e+03 -3.60041967e+02  7.39387529e+02  3.78622307e+03
  1.90822745e+03 -1.01260890e+03 -8.44915990e+02  3.90474504e+02
  8.93670948e+01 -4.13370160e+02  3.17318670e+02  2.62142353e+02
 -1.97467523e+03 -8.33784811e+01  1.37018053e+03  1.98120906e+01
  1.06851540e+02 -1.24268543e+03  5.19657177e+03  3.99725561e+02
 -1.59813419e+03 -1.41098539e+03 -9.83985644e+02 -5.49910608e+02
  1.76826426e+03  8.19449883e+02 -9.81397495e+01  5.05031968e+03
 -1.23676564e+03 -2.33667438e+03 -6.07563734e+02 -4.44162459e+02
  1.32807378e+03 -8.37139525e+02  2.58677837e+02 -2.99574959e+02
  9.54991876e+02 -1.11615934e+03  6.00441675e+02 -1.93837429e+02
  4.04445819e+02 -1.36460219e+02  5.11625670e+02  7.75800409e+02
 -1.06682620e+03 -3.72172632e+02 -8.82283079e+02  9.39131218e+02
  9.44436533e+02  3.00056610e+02 -8.98935670e+02  2.34773836e+02
 -4.36336571e+02  3.05606973e+01 -3.05606973e+01  2.82006427e+02
 -6.48860553e+02 -3.09823547e+02  0.00000000e+00  1.15205414e+02
  4.22995710e+03  8.17279649e+02 -1.00060078e+03 -1.41954367e+03
 -4.57101161e+02 -1.13535585e+03 -1.33596051e+03  1.02460536e+02
 -3.33596500e+02 -1.50801908e+03  1.42960920e+03 -3.09138371e+02
 -5.88057507e+02 -4.91730320e+02  3.44939837e+02  7.41209822e+02
 -7.95181344e+02 -2.01631889e+02  3.93050114e+02 -4.26626321e+02
  4.36629743e+02  3.84642273e+02 -9.20190822e+01 -9.04083057e+02
  9.00030911e+02  6.48581245e+03 -8.70019372e+02 -2.52348691e+02
 -1.82446024e+03 -1.48782190e+02 -5.16955491e+03 -2.59291892e+02
  1.73998599e+02  1.35381376e+03  6.61517053e+02  4.91802795e+02
 -6.16944117e+02 -1.11992154e+02 -1.81920110e+03 -4.44060473e+02
  1.60146639e+03 -4.26783910e+02  2.89954277e+01  4.89499355e+02
```

```
-1.82446024e+03 -1.48782190e+02 -5.16955491e+03 -2.59291892e+02
 1.73998599e+02  1.35381376e+03  6.61517053e+02  4.91802795e+02
-6.16944117e+02 -1.11992154e+02 -1.81920110e+03 -4.44060473e+02
 1.60146639e+03 -4.26783910e+02  2.89954277e+01  4.89499355e+02
-6.69485133e+02 -9.71136534e+02 -5.56438320e+01  9.31199944e+02
-6.27781092e+02  7.82567719e+01  9.12464480e+02  1.44892028e+03
 9.81610077e+02 -8.14844154e+02  6.57357420e+02  3.69285032e+02
 2.14619686e+02 -8.68198894e+02 -1.84939000e+03  7.76873940e+02
 1.56781357e+03 -9.42388535e+02 -6.13627222e+02  2.25929253e+03]
```

9. Prediction:

```
[13]
✓ 0s
y_pred = model.predict(X_test)
comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})
print(comparison.head(10))
```

```
   Actual      Predicted
0  154500  156799.660184
1  325000  348262.803548
2  115000   94014.158247
3  159000  176502.277239
4  315500  330385.909715
5   75500   67397.776414
6  311500  237191.219780
7  146000  146566.763374
8   84500   58103.632675
9  135500  152604.707323
```

10. R2 Score:

```
[14]
✓ 0s
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

```
R2 Score: 0.882215933343812
```

11. Save the Model

```
[15]
✓ 0s
joblib.dump(model, "LR_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(list(X.columns), "columns.pkl")

print("Model Saved Successfully!")
```

```
Model Saved Successfully!
```

Summary

- Loaded the House Prices (Ames Housing) dataset.
- Handled missing values using median (numerical) and mode (categorical).
- Separated the dataset into independent features (X) and target variable (SalePrice).
- Applied One-Hot Encoding to convert categorical features into numerical format.
- Scaled the numerical features using StandardScaler.
- Split the dataset into training and testing sets.
- Trained a Linear Regression model on the training data.
- Made predictions on the test data and evaluated the model using the R^2 Score.
- Saved the trained model, scaler, and feature columns using joblib for future use.